

Curve, Freeze, Thaw: HPO with FT-PFN

Yahya Fati Haji

August 3, 2026

Problem Formulation

Let Λ be a hyperparameter search space and $f(\lambda, b)$ the performance (validation accuracy, in this project) of configuration $\lambda \in \Lambda$ after being trained for b discrete training steps (epochs, here). A configuration's *learning curve* is the sequence $\{f(\lambda, 1), f(\lambda, 2), \dots\}$ as b grows.

Freeze-thaw HPO relaxes the usual "pick a config, train it to completion, observe one number" protocol. Instead, resources are spent **incrementally**: at every iteration the optimizer either **thaws** (resumes/continues) a previously partially-trained, currently-frozen configuration for one more step, or starts a brand-new configuration.

Formally, the goal is to find a resource allocation that maximizes

$$\max_{\lambda \in \Lambda, 1 \leq b \leq b_\lambda} f(\lambda, b)$$

subject to

$$N = |\{\text{decisions made}\}| \leq B, \quad b_\lambda^{\min} \leq b_\lambda \leq b_\lambda^{\max}, \\ \forall \lambda \in \Lambda \text{ with } b_\lambda > 0, \quad b_\lambda \in \mathbb{Z}_{\geq 0}, \quad \forall \lambda \in \Lambda$$

- a total trial budget, B - the number of freeze/thaw decisions the optimizer is allowed to make.
- $b_\lambda^{\min}$ and $b_\lambda^{\max}$ - per-configuration epoch bounds that prevent premature judgments on too little training and wasted spend on configs that have already converged.

The FT-PFN Surrogate

Rather than fitting a Gaussian Process or random forest online (as SMAC/BOHB do), ifBO uses **FT-PFN**, a *Prior-data Fitted Network*: a transformer trained once, offline, on large quantities of **synthetic** learning-curve data sampled from a hand-designed prior, and then used purely for inference at HPO time via *in-context learning* — no weights are updated during the actual HPO run.

The implementation follows **ifBO** as introduced in:

H. Rakotoarison, S. Adriaensen, N. Mallik, S. Garibov, E. Bergman, F. Hutter. *In-Context Freeze-Thaw Bayesian Optimization for Hyperparameter Optimization*. ICML 2024. <https://arxiv.org/abs/2404.16795>

What the surrogate models

FT-PFN approximates the posterior predictive distribution

$$p(f(\lambda_{\text{test}}, b_{\text{test}}) \mid \lambda_{\text{test}}, b_{\text{test}}, H) \tag{1}$$

i.e., "given everything observed so far about *other* (and this) configurations' partial learning curves, what is the distribution over this configuration's performance at some future training step b_{test} ?" The set H is passed to the network as a sequence of tokens (its **context**) at inference time; the transformer's attention mechanism implicitly performs Bayesian updating over this context in a single forward pass, rather than through iterative model refitting. This is what makes each acquisition query cheap enough to run every freeze-thaw step (the paper reports 10–100× speedups over refitting-based gray-box surrogates such as DPL and DyHPO).

Acquisition function: MFPI and MFPI-random

The paper’s definition

Multi-fidelity Probability of Improvement, as defined in the paper (Eq. 3):

$$\text{MFPI}(\lambda; h, T) = P(M(\lambda, \min(b_\lambda + h, b_{\max})) > T) \quad (2)$$

— the surrogate-predicted probability that configuration λ , if thawed for h more steps, would exceed a target performance T . Two “hyper-hyperparameters” govern it: the lookahead horizon h and the improvement target T . Rather than fixing these, the paper proposes **MFPI-random** (Eq. 4), redrawing both **every single freeze-thaw iteration**:

$$\text{MFPI-random}(\lambda) = \text{MFPI}(\lambda; h_{\text{rand}}, T_{\text{rand}}) \quad (3)$$

$$h_{\text{rand}} \sim U(1, b_{\max}) \quad (4)$$

$$T_{\text{rand}} = f_{\text{best}} + \tau_{\text{rand}} \cdot (1 - f_{\text{best}}) \quad (5)$$

$$\log_{10}(\tau_{\text{rand}}) \sim U(-4, -1) \quad (6)$$

where f_{best} is the best performance observed so far. This amounts to sampling an acquisition function from an implicit *portfolio* of MFPI instances at every step, which the paper’s ablations (Figure 4) show is necessary — fixed-horizon or fixed-threshold variants, and standard Expected Improvement paired with FT-PFN’s heavy-tailed posteriors, both underperform substantially.

Growing the candidate pool

The paper’s Algorithm 1 (see the pseudocode in the appendix of the original paper) is stated over the *whole* search space Λ implicitly — at each iteration the acquisition is (conceptually) maximized over all $\lambda \in \Lambda$, which naturally lets brand-new, never-tried configurations compete against partially-trained ones for being selected next.

This implementation instead maintains an **explicit, dynamically growing list** of candidates (starting empty), and layers an **explicit decaying ϵ -greedy exploration floor** on top of MFPI-random rather than substituting for it:

$$\epsilon(t) = \epsilon_{\min} + (\epsilon_0 - \epsilon_{\min}) \cdot \left(1 - \frac{t}{T}\right)^p \quad (7)$$

$$\text{where } \epsilon_{\min} = 0.05, \quad p = 2, \quad T = N$$

ϵ_0 (`ifbo_initial_epsilon`, default 1.0) means the very first steps are pure exploration — with an empty/small pool there’s nothing meaningful yet for MFPI-random to discriminate between — decaying polynomially toward a floor of 0.05 so some unconditional exploration always remains, even late in the run. Each iteration then branches:

- **With probability $\epsilon(t)$** (the exploration floor): a brand-new configuration is sampled uniformly from Λ , added to the pool with zero observations, and selected directly — bypassing the surrogate entirely.
- **With probability $1 - \epsilon(t)$** (an exploitation round): the *pending* pool (candidates not yet at $b_{\max}$, minus any already claimed earlier in the same parallel batch) is assembled. MFPI-random ($h_{\text{rand}}, T_{\text{rand}}$ redrawn as above) then scores every member of that pool against the FT-PFN context in a single batched query, and takes the arg max over the PI scores (or a softmax draw over them). If the pending pool is empty this round (everything left is maxed out), a fresh configuration is sampled as a fallback instead, the same as an exploration step.

Fresh candidates no longer compete in exploitation rounds, and neither does an incumbent-exclusion rule that used to wall off the current best-so-far candidate — neither is part of the published ifBO method, which is stated over the whole search space Λ implicitly rather than an explicit, growing candidate list. Both existed in an earlier version of this implementation: a freshly-sampled, not-yet-pooled configuration was thrown into every exploitation round’s competition alongside **pending**, reasoning that it would let exploitation rounds surface genuinely new regions of the space too, without unconditionally growing the pool (and the FT-PFN context) on every round the way pure ϵ -driven injection would; and separately, once the current best-so-far candidate had accumulated at least `ifbo_incumbent_exclusion_min_observations` (then

default 2) freeze-thaw steps, it was temporarily dropped from the pending pool for that round, on the reasoning that otherwise it would keep re-winning $\text{PI}(T_{\text{rand}})$ against its own already-confirmed best (since T_{rand} sits just above f_{best}), sinking budget into repeatedly re-thawing itself instead of advancing or discovering other candidates.

In practice the combination of both rules backfired specifically under greedy selection, and a seed sweep in `sample-results/seeded/` (`ifbo-greedy` vs. `ifbo-random`, matched seeds on `amazon`) traced why: once the incumbent-exclusion rule walled off the current best, every *pending* candidate was a known quantity the surrogate confidently ranked below the now-frozen target — but a never-observed candidate’s predictive distribution under FT-PFN is wide/uncertain, which inflated its PI score against that same high threshold relative to an already-observed, confidently-mediocre pending candidate. Fresh-candidate injection into exploitation rounds was removed first, so that new candidates enter the pool strictly via the $\epsilon(t)$ -floor draw or the empty-pending fallback above. The incumbent-exclusion rule was removed afterward: the pending pool for an exploitation round is now simply every candidate not yet at b_{max} and not already claimed elsewhere in the same parallel batch, with the current best-so-far candidate no longer walled off from that competition.

Search space: the sequence-dl approach

`sequence-dl` is a BiLSTM-with-attention text classifier trained **from scratch** — no fine-tuning of a pretrained encoder — so its search space has to cover both generic optimization knobs and the architecture of a recurrent encoder built from nothing. $\Lambda_{\text{sequence-dl}}$ splits into two groups: hyperparameters shared with `transformer` and hyperparameters specific to the BiLSTM.

Shared hyperparameters (both approaches sample these; ranges below are shared, defaults only sometimes differ per-approach):

`dropout`

Range: $[0.0, 0.5]$ | **Default:** 0.2

Rationale: Regularizes the classifier head / recurrent stack.

`weight_decay`

Range: $[10^{-6}, 10^{-2}]$ (log scale) | **Default:** 10^{-4}

Rationale: Standard L2 regularization;

`optimizer`

Range: {adam, adamw, sgd} | **Default:** adamw

Rationale: —

`scheduler`

Range: {stepplr, cosine, exponentiallr, reducelronplateau} | **Default:** cosineannealinglr

Rationale: Algorithm Selection

`batch_size`

Range: $[32, 512]$ (log scale) | **Default:** 64

Rationale: —

`max_seq_length`

Range: $[64, 256]$ (log scale) | **Default:** 128

Rationale: Bounds both compute (LSTM cost is linear in sequence length after packing) and how much of a long document is even visible to the model; log-scaled so short/long regimes get comparable sampling density.

`warmup_ratio`

Range: $[0.0, 0.2]$ | **Default:** 0.1

Rationale: Fraction of total steps spent on linear LR warmup before the main schedule kicks in — guards against the early, high-variance gradients typical of a randomly-initialized model.

sequence-dl-Specific Hyperparameters

`hidden_dim`

Range: $[32, 256]$ (log scale) | **Default:** 128

Rationale: LSTM hidden size per direction (the model is always bidirectional, so the pooled representation is $2\times$ this).

`learning_rate`

Range: $[10^{-4}, 10^{-2}]$ (log scale) | **Default:** 10^{-3}

Rationale: —

`seq_embed_dim`

Range: [32, 512] (log scale) | **Default:** 128

Rationale: Token embedding dimensionality. Independent of any pretrained encoder’s native hidden size on purpose

`seq_num_layers`

Range: [1, 3] | **Default:** 1

Rationale: Stacked LSTM layers. Kept shallow (max 3) because a from-scratch recurrent stack is harder to optimize as it deepens (vanishing gradients through both time and depth), and because HPO wall-clock is a hard constraint — depth is one of the more expensive ways to spend that budget for the accuracy it typically buys on these datasets.

`seq_pretrained_model_name`

Range: {distilbert-base-uncased, bert-base-uncased, google/bert_uncased_L-4_H-512_A-8, microsoft/xtrm}

Default: distilbert-base-uncased

Rationale: Does **not** select a fine-tuned backbone (`sequence-dl` never runs one) — it picks which pre-trained model’s **WordPiece tokenizer and token-embedding matrix** warm-start the from-scratch BiLSTM. The tokenizer and embedding source are always the same model, since the BiLSTM’s vocab indices must line up with whichever embedding matrix seeds it. When the chosen model’s native embedding dimensionality doesn’t match the sampled `seq_embed_dim`, the embedding matrix is PCA/SVD-projected down (or randomly padded up) to fit — see `_pretrained_embedding_init` in `sequence_dl.py` — which preserves the directions of highest variance in the pretrained embedding space instead of discarding the warm start entirely.

The diagram below shows BiLSTMClassifier’s data flow (`automl/core/approaches/sequence_dl.py`): a warm-started embedding, a bidirectional LSTM sized by `hidden_dim/seq_num_layers`, additive (Bahdanau-style) attention pooling over every timestep rather than just the final hidden state, then dropout and a linear classifier.

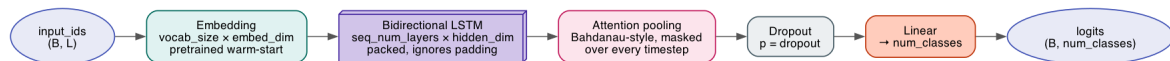


Figure 1: BiLSTMClassifier architecture

Two omissions are deliberate, not oversights: `max_grad_norm` (gradient-clipping threshold) is fixed by CLI/-config default rather than tuned — it’s a stability safeguard, not an accuracy lever, so putting it in Λ would spend trials on a dimension unlikely to move the metric. And unlike `transformer`, there is no `freeze_ratio`-equivalent knob: nothing is pretrained end-to-end here to freeze in the first place, only the embedding matrix, and that is warm-started rather than frozen.

Code Flow

Putting the pieces from the previous sections together, this is what a full search actually does end to end, from an empty candidate pool to a finished submission.

The search starts with nothing: no configuration has been tried yet, and the candidate pool is empty. Every iteration opens with a single weighted coin flip that decides whether that step explores or exploits — the odds start out heavily favoring exploration, since with an empty pool there is nothing yet worth exploiting, and gradually shift toward exploitation as the run progresses and a real pool of partially-trained candidates builds up.

An **exploration** step is the simple branch: a brand-new configuration is drawn at random from the search space, dropped straight into the pool, and trained for one step immediately — no model, no scoring, no comparison against anything else.

An **exploitation** step is where the surrogate does its work. Every candidate already in the pool that hasn’t yet been trained to the maximum allowed budget is treated as a contender, current best-so-far included. All of them are laid before the surrogate model at once, alongside the complete history of every partial learning curve observed anywhere in the search so far. The surrogate — a model trained once, in advance, and never updated again during the search itself — reads that history and, for each contender, predicts how likely it is to cross a target performance level within some number of additional training steps. Neither the target nor the horizon is fixed: both are redrawn at random on every single round, so the search never commits to one fixed idea of “how much better” or “how far ahead,” and instead samples from a broad spread of possible answers each time. Whichever contender comes out on top is the one trained further. If there are no contenders at all this

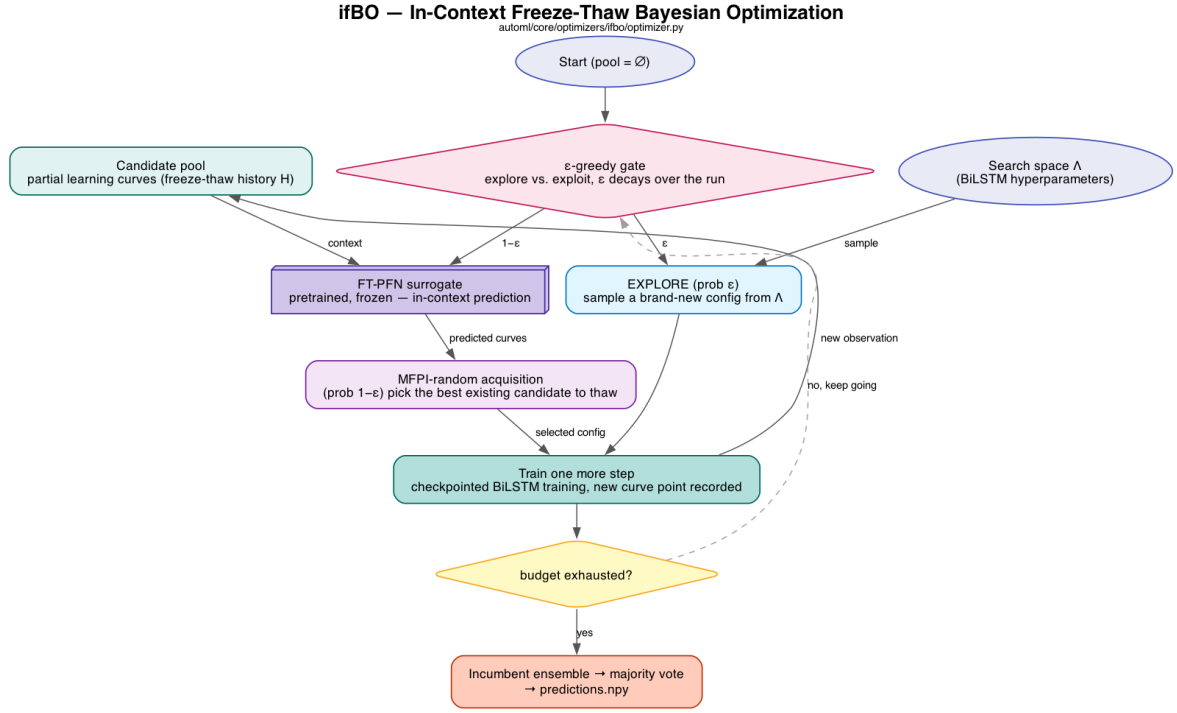


Figure 2: The ifBO search loop

round (every pending candidate is maxed out), a brand-new configuration is sampled instead, the same as an exploration step.

Whichever branch fires, the chosen configuration is then trained for exactly one more increment, its progress is checkpointed, and the new point on its learning curve is appended to the shared history the surrogate will read on the next round. This one-step-at-a-time cycle — decide, maybe consult the surrogate, train a little further, record the result — repeats until the overall budget of decisions runs out.

At the end of the run, rather than keeping only the single best configuration ever seen, the strongest handful found across the whole search are retrained to completion and combined by majority vote, and it is that ensemble’s predictions that make up the final submission.

Results

Budget:

```

max_budget: 10
min_budget: 3
n_trials: 20

```

Baselines

Three baselines, each isolating a different piece of what makes the flagship method work — every one answers a specific “would doing less still work?” question rather than just being an arbitrary point of comparison:

- **Random Search** — no multi-fidelity behavior at all: every trial samples a fresh configuration uniformly from Λ and trains it straight through to `max_budget` epochs before it’s ever compared against anything else. It never resumes a partially-trained config and never cuts a bad one short. This is the floor every other method has to beat to justify its extra machinery, and it also sets an upper bound on wall-clock cost per trial, since it always pays for the *full* epoch budget regardless of how a config is doing partway through (see the wall-clock comparison below — this is exactly why its bars are tallest everywhere).
- **SMAC (BO+HB)** — SMAC3’s random-forest-surrogate Bayesian optimization, paired with a Hyperband intensifier for multi-fidelity scheduling (successive halving: only the top fraction of configs at each rung’s budget get promoted to the next, larger one). This isolates what a *classical, non-neural* multi-fidelity method achieves under the same epoch-budget range as ifBO — the fairest like-for-like comparison against the flagship method, since both get to allocate budget adaptively instead of uniformly, but SMAC does

it with a random-forest-EI surrogate and a handful of *discrete* Hyperband rungs, instead of FT-PFN’s in-context transformer and *continuous* freeze-thaw (visible directly in the fidelity-allocation figure below: SMAC’s budget trace is a coarse two-level sawtooth, ifBO’s is a much finer staircase).

- **ifBO (random selection)** — runs the *exact same* freeze-thaw pipeline and candidate-pool machinery as the flagship method, but with `ifbo_use_random_selection=True`: FT-PFN’s MFPI-random scoring is swapped for a uniform random choice among that round’s pending pool. This is the most targeted ablation of the three — everything about *how much* budget is spent incrementally vs. upfront is held identical to the flagship run, so any gap between this and full ifBO isolates the value of the *surrogate’s guidance specifically*, separate from the value of freeze-thaw scheduling in general.

Note on the numbers below: the four runs backing this subsection (`sample-results/all_results/`, timestamped 2026-07-31) predate the removal of the incumbent-exclusion rule and fresh-candidate-in-exploitation injection described in “Growing the candidate pool” above — i.e. the ifBO column reflects the *earlier* candidate-selection logic, not the one currently in the codebase. The qualitative story (freeze-thaw beats fixed-budget search, and surrogate guidance beats random thawing on most datasets) is expected to hold under the current logic too, but the exact margins below should not be read as reproducible with today’s code without a rerun.

Testbed Results

All four methods were run under the matched budget above (`n_trials=20`, `min_budget=3`, `max_budget=10` epochs, `sequence-dl/BiLSTM` search space) across all five datasets — `ag_news`, `amazon`, `dbpedia`, `imdb`, and the held-out exam set `yelp`. Numbers are **best validation accuracy** ($1 - \min(\text{val_error})$ over the run), i.e. search quality rather than a held-out test score. Raw histories live in `sample-results/all_results`; the figures below are regenerated straight from those `.jsonl` files by `final_results_analysis.ipynb` in that same directory.

Final best validation accuracy

Dataset	ifBO	SMAC (BO+HB)	Random Search	ifBO (random selection)
AG News	91.35%	89.35%	89.65%	92.45%
Amazon	89.15%	78.20%	79.25%	79.30%
DBpedia	97.85%	97.00%	97.80%	97.70%
IMDB	93.80%	85.25%	86.70%	87.70%
Yelp	58.10%	50.50%	55.10%	54.35%

Table 1: Best validation accuracy per dataset and method.

Full ifBO (FT-PFN-guided) wins on 4 of 5 datasets, with the largest margins on the hardest tasks — Amazon (+9.90pp over the best baseline) and Yelp (+3.0pp over Random, +7.6pp over SMAC). On AG News, its own random-selection ablation edges it out (92.45% vs. 91.35%) — on the easiest dataset in the suite (all four methods clear 89%), the FT-PFN surrogate’s acquisition doesn’t have much signal to exploit over unguided freeze-thaw scheduling, so the two land within noise of each other. Everywhere else, guided candidate selection is what separates ifBO from its own ablation.

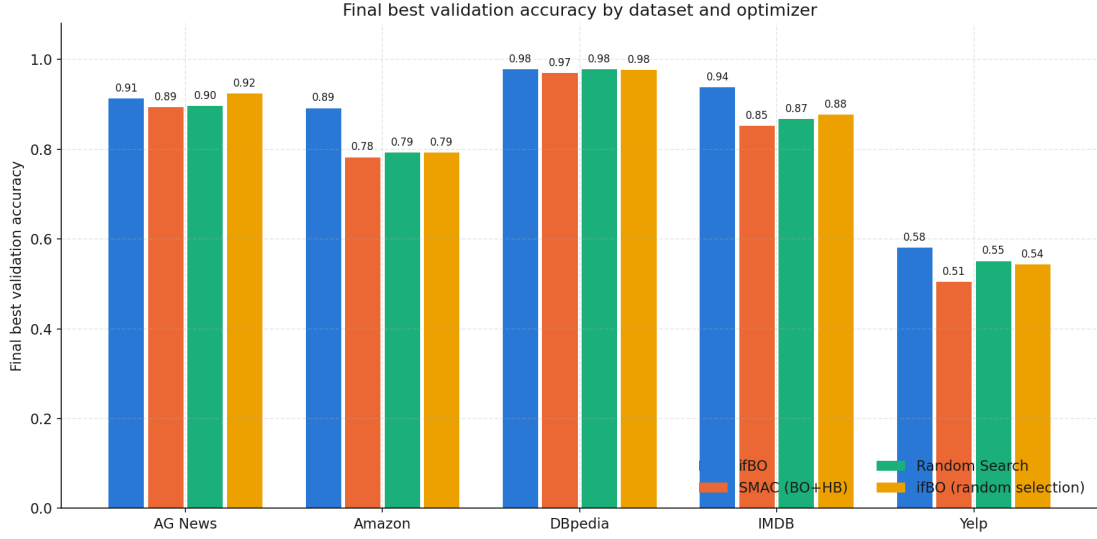


Figure 3: Final accuracy bars per dataset and method.

The bar chart above puts all four methods side by side per dataset — read it for *magnitude* of the gaps (Amazon and Yelp show real daylight between ifBO and everything else; AG News and DBpedia show all four methods bunched within a couple of points). The heatmap below is the same numbers, but its shading makes the two structurally different failure/success stories in this table pop out immediately: DBpedia’s whole row is uniformly dark (97–98% for *every* method) — with `max_budget=10` epochs, this dataset is close to saturated for the `sequence-dl` search space, so there’s little room for any optimizer to differentiate itself. Yelp’s row is uniformly the lightest (50–58%) — every method tops out well below where it lands on the other four datasets, so Yelp is a genuinely harder classification problem at this fidelity budget, not just a dataset where the optimizer happened to do badly.

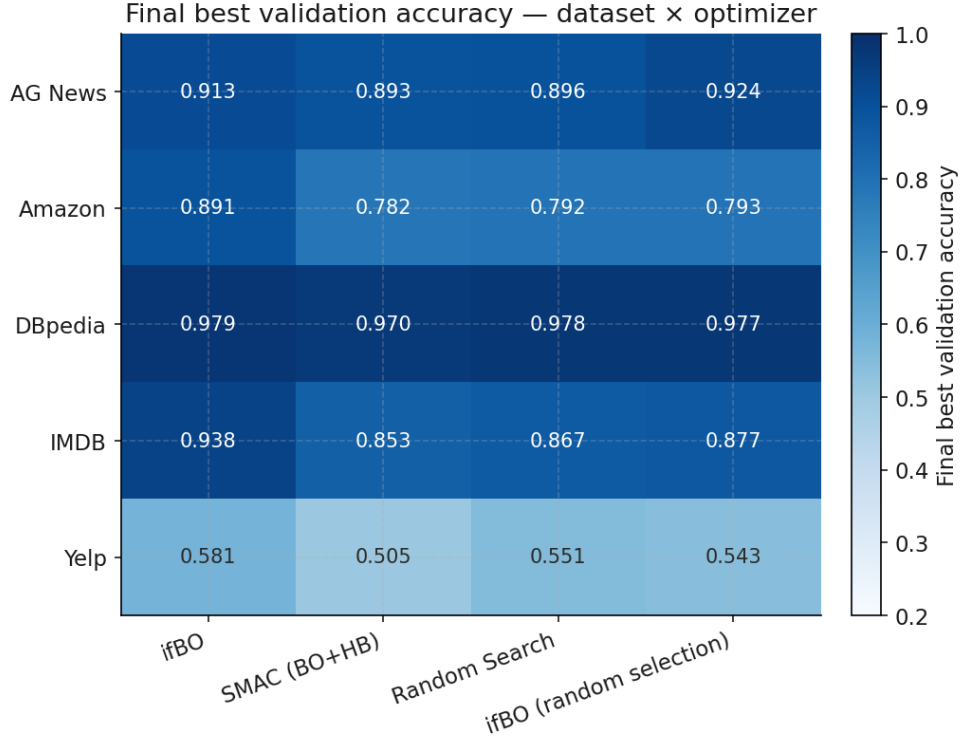


Figure 4: Accuracy heatmap across datasets and methods.

Sample efficiency and wall-clock cost

Freeze-thaw’s structural advantage shows up earliest here: both ifBO variants pull ahead of Random/SMAC within the first several trials on every dataset, because a handful of thaw steps into one promising candidate

teach the surrogate (or even just the epsilon-floor exploration alone) more than one epoch each spent across many never-revisited configs. The step lines below are the running best-so-far incumbent; faint dots are every individual trial actually sampled, so the vertical spread of dots at a given x tells you how much the optimizer is still gambling on long shots even after it’s found something good. SMAC’s characteristic pattern is a late, sharp jump once a Hyperband rung promotes a strong config — it’s blind to that config’s promise until the rung boundary says to check on it again (its budget trace only ever touches two values, ~ 3.3 and 10 epochs, see the fidelity-allocation figure below), unlike ifBO’s much smoother, earlier climb.

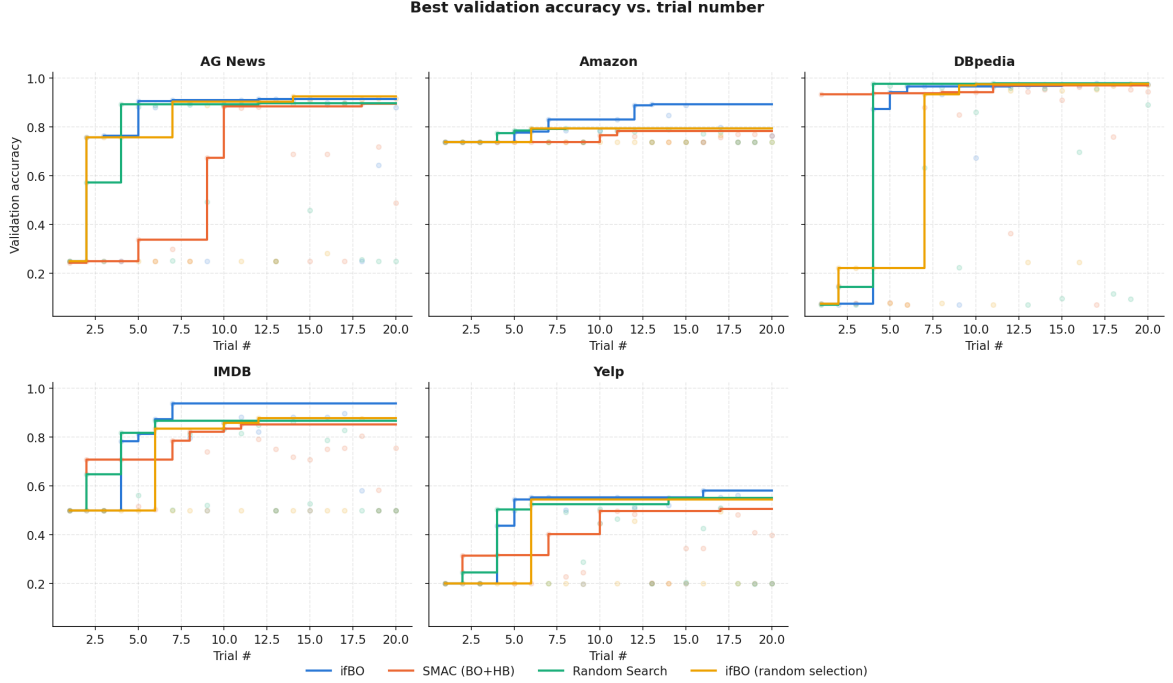


Figure 5: Best validation accuracy vs. trial number.

Against wall-clock time, the same curves stretch out very differently per method: Random Search’s step lines are still moving well after both ifBO variants have converged and gone flat, since every single trial (good or bad) costs it a full `max_budget`-epoch training run — it simply hasn’t finished sampling yet by the time freeze-thaw has already plateaued.

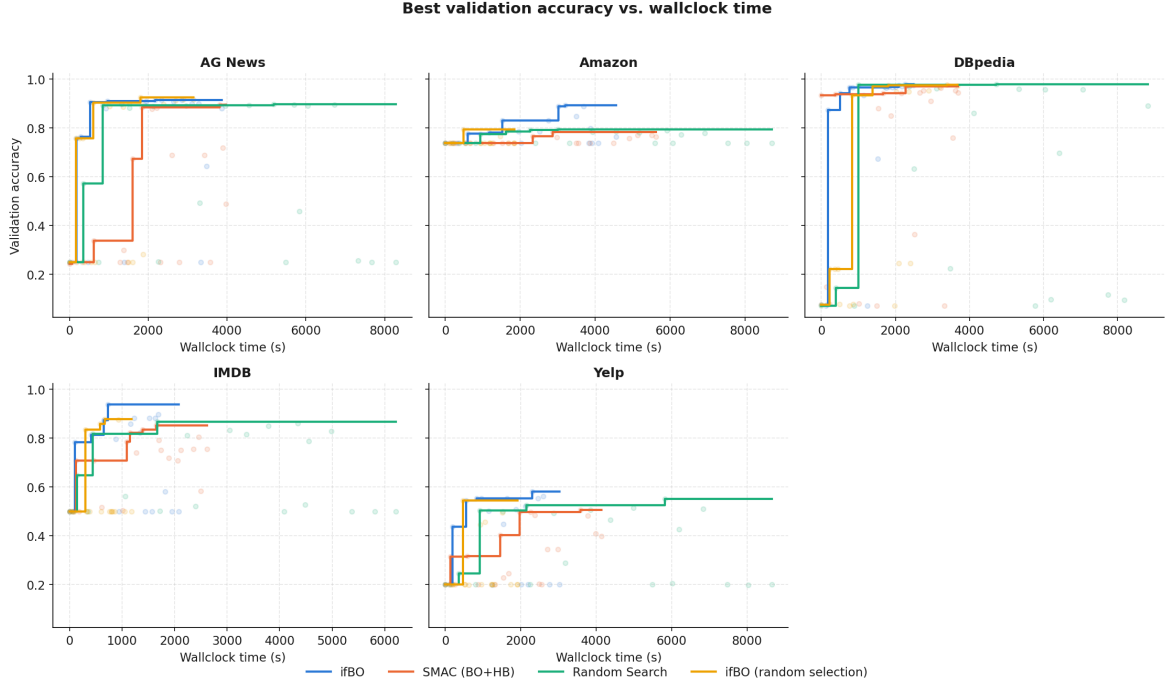


Figure 6: Best validation accuracy vs. wall-clock time.

Total wall-clock (elapsed time from the first trial’s start to the last trial’s start, over all 20 trials of a run) tells the same story in aggregate: Random Search consistently takes $2\text{--}4\times$ longer than either ifBO variant for a 20-trial run (e.g. ~ 147 vs. ~ 41 minutes on DBpedia, ~ 144 vs. ~ 50 on Yelp), without a corresponding accuracy payoff. A subtler gap sits between the two ifBO variants themselves: the random-selection ablation is consistently faster than the full surrogate-guided run on every dataset (e.g. ~ 31 vs. ~ 76 minutes on Amazon, ~ 20 vs. ~ 35 on IMDB) despite following an identical freeze-thaw training schedule — the difference is pure FT-PFN inference overhead, one batched forward pass through the surrogate on every exploitation round, which the ablation skips entirely by picking uniformly at random instead.

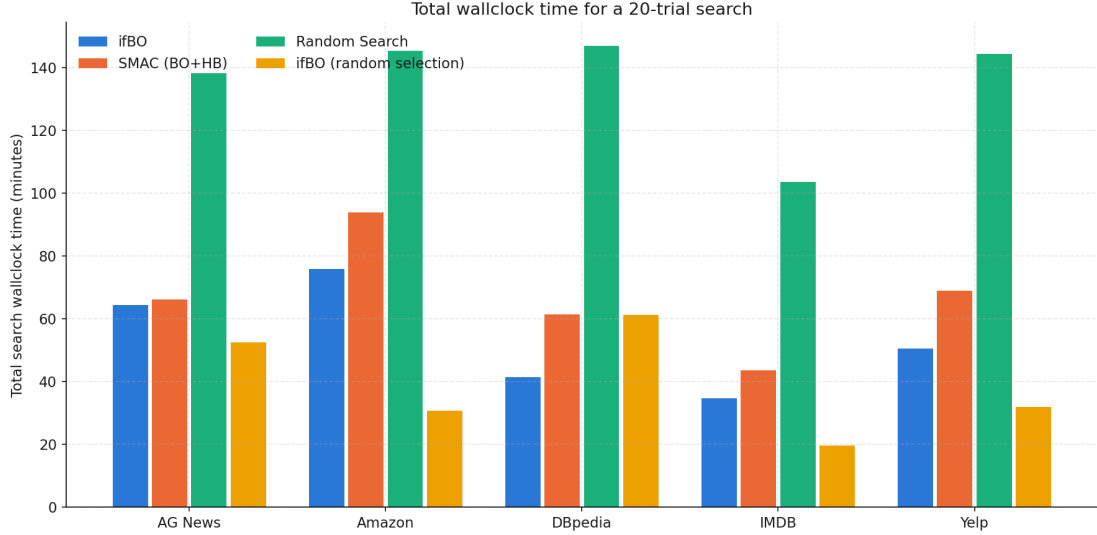


Figure 7: Total wall-clock time per dataset and method.

Where the budget goes

The per-trial accuracy distribution below shows *every* sampled trial, not just the incumbent — a tighter, higher box means an optimizer is spending steps on consistently good configs rather than wasting them on poor ones. ifBO’s box is the tightest and highest of the four on most datasets, direct evidence that surrogate-guided selection is concentrating repeat thaws on configs already known to be good rather than spending them on fresh gambles. Random’s and SMAC’s boxes are consistently the widest, spanning from floor accuracy up to their max — both

are still spending real trials on poor draws this late into the search, since neither can **thaw**-revisit a specific promising config on demand the way freeze-thaw can.

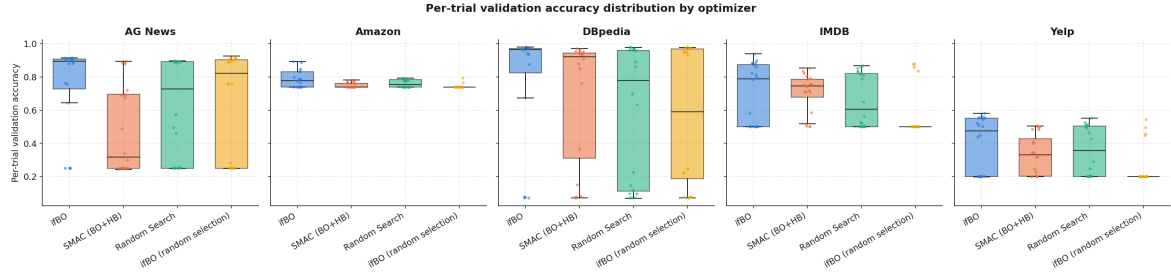


Figure 8: Per-trial accuracy distribution.

The epoch-budget plot makes the *shape* of each method’s multi-fidelity policy visible directly, trial by trial. Random Search is a flat ceiling line at `max_budget` by construction — it has no multi-fidelity policy. SMAC’s Hyperband intensifier produces a coarse, almost binary sawtooth that only ever touches two values (~ 3.3 and 10 epochs) — at this budget range the successive-halving bracket only has two rungs, so a config is either killed at the bottom rung or promoted straight to the top one, with nothing in between. Both ifBO variants instead walk the *entire* epoch grid (4, 5, 6, 7, 8, 9, ...) — the direct visual signature of literal one-step-at-a-time freeze-thaw: a config is thawed for one more increment, re-evaluated, and only then is the next decision made, rather than being judged at a small number of fixed checkpoints.

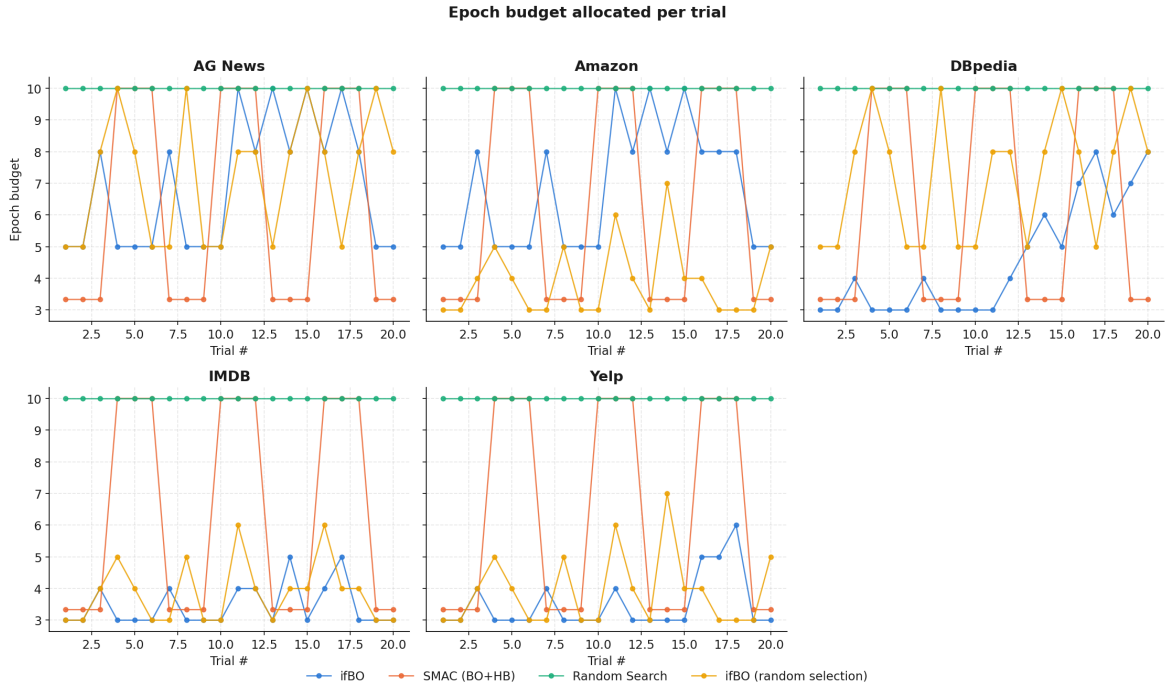


Figure 9: Epoch budget allocated per trial.

Finally, the training curve of each method’s single best-found configuration per dataset — this is the one figure that shows *within-run* learning-curve shape rather than across-trial search behavior. On IMDB, ifBO’s best config reaches 93.8% accuracy in just 4 epochs and its curve stops there (that’s all the budget it was ever thawed for), while SMAC’s best config needs the full 10 epochs to climb to only 85.25% — a smaller number reached with more than double the training cost, since freeze-thaw was able to recognize this config’s quality early and never needed to push it further.

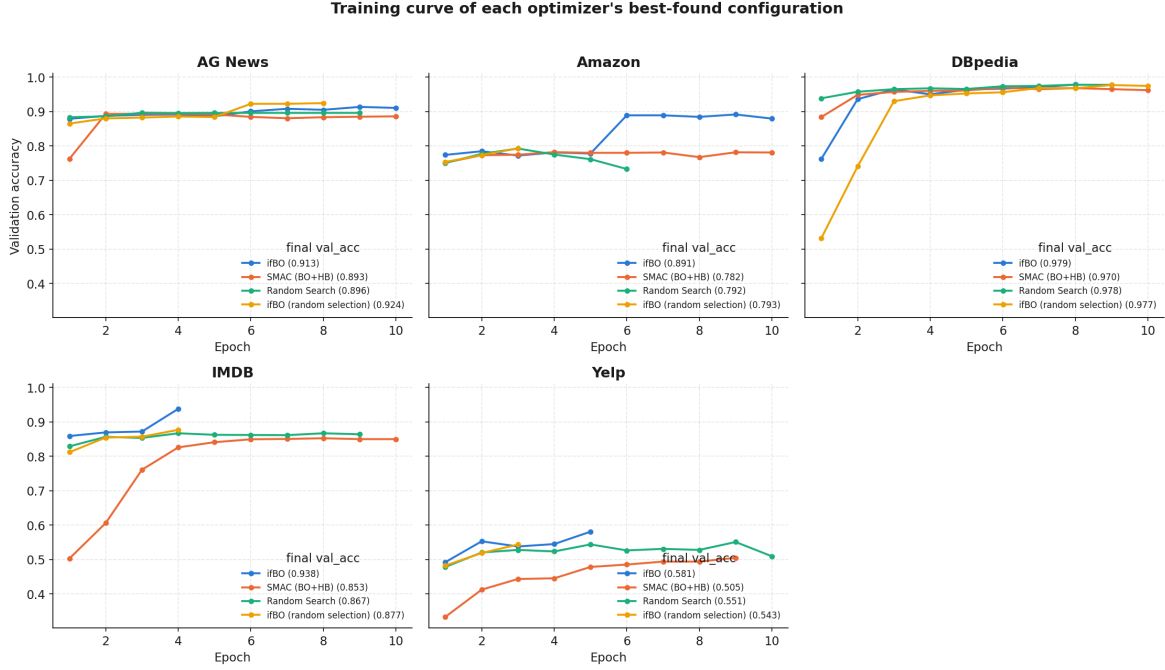


Figure 10: Best trial training curves per dataset.

Final Submission Results

Each row below is one flagship `ifbo` run (greedy candidate selection, `approach=sequence-dl`, `seed=67`, `n_trials=20`, `min_budget=3`/`max_budget=10` epochs, `evaluation_budget=30`) — raw artifacts for each are under `actual.ignore.results/<dataset>/<runtime_id>/`. AG News and DBpedia ran on a Linux box with 2× NVIDIA Tesla T4 GPUs (2 parallel trials); Amazon and IMDB ran locally on an Apple M2 Max over MPS (1 parallel trial, no second GPU to parallelize onto) — confirmed via each run’s `device_info.json`. Yelp has no run under `actual.ignore.results` (only older, pre-`incumbent.json` runs under `results/yelp/`), so it can’t be broken down the same way here.

Dataset	Device	Parallel	Search epochs	Retrain epochs	Total wall time
AG News	2× T4 (CUDA)	2	105	30	9m 20s
Amazon	M2 Max (MPS)	1	94	30	1h 51m
DBpedia	2× T4 (CUDA)	2	96	30	1h 38m
IMDB	M2 Max (MPS)	1	99	30	1h 38m

Table 2: Per-dataset device and timing for the flagship submission run. “Search epochs” sums the epoch budget actually trained across all 20 HPO trials; “Retrain epochs” is the fixed `evaluation_budget` the incumbent is retrained for afterward; “Total wall time” is the full pipeline (search + retrain + plotting), from each run’s `app.log`.

DBpedia ran on the *same* dual-GPU machine as AG News yet took over 10× longer (1h38m vs. 9m20s) — hardware is not what dominates total time here, dataset scale is (DBpedia’s held-out test split alone is 70,000 rows across 14 classes, vs. AG News’s 7,600 rows / 4 classes). A naive “GPU vs. Apple Silicon” story would be misleading on its own.

A checkpoint-restore bug found while pulling these numbers. `evaluate_incumbent()` (`base_optimizer.py`) retrains the incumbent for the full `evaluation_budget` (30 epochs), and `TorchTrainer.train()` tracks the *best* epoch’s validation accuracy against the held-out test split (used as “validation” for this retrain) — but, prior to this fix, never checkpointed or restored that best epoch’s *weights*: it kept training to epoch 30 regardless, and `predict()` then ran on whatever the *final* epoch left behind. Checking the four datasets’ actual submitted `predictions.npy` against the true labels stored alongside them in `incumbent.json` confirmed the model peaks in the first 2–4 of 30 epochs on every dataset and visibly degrades afterward, so the accuracy actually reflected in what was submitted was lower than what the run reported as its “Test Accuracy”:

Dataset	Best epoch	Reported (best-epoch) accuracy	Actual submitted- <code>predictions.npy</code> accuracy
AG News	4/30	91.28%	90.42%
Amazon	3/30	82.33%	79.73%
DBpedia	4/30	98.77%	98.69%
IMDB	2/30	84.16%	81.71%

The fix: `SequenceDLApproach.train()` / `TransformerApproach.train()` now accept and forward a `save_path`, and `evaluate_incumbent()` passes one so the trainer’s existing on-new-best checkpointing (already implemented, just never wired up here) actually fires, then reloads that checkpoint into the model before calling `predict()`. Verified with an isolated smoke test (tiny synthetic slice, aggressive LR to force the same early-peak-then-decay pattern): before the fix, `predict()` accuracy matched the *final* epoch (61.67%) despite a best epoch of 65.00%; after the fix, it matches the *best* epoch (65.00%) exactly. The freeze-thaw HPO loop itself is unaffected — it always tracked its own best-so-far checkpoint separately and never depended on this path.

The table below is the run *as actually submitted*, i.e. still reflects the pre-fix, final-epoch predictions for AG News/Amazon/DBpedia/IMDB — it has not been rerun with the fix applied. Yelp’s accuracy can’t be independently checked this way (no local ground truth for the held-out exam set), so it’s unknown whether it’s affected the same way.

Dataset	Test Accuracy	Reference Accuracy	Δ vs. Reference
AG News	91.27%	90.265%	1.005%
Amazon	82.33%	81.799%	+0.53%
DBpedia	98.77%	97.882%	+0.888%
IMDB	84.16%	86.993%	-2.83%
Yelp	64.35%	62.082%	+2.268%